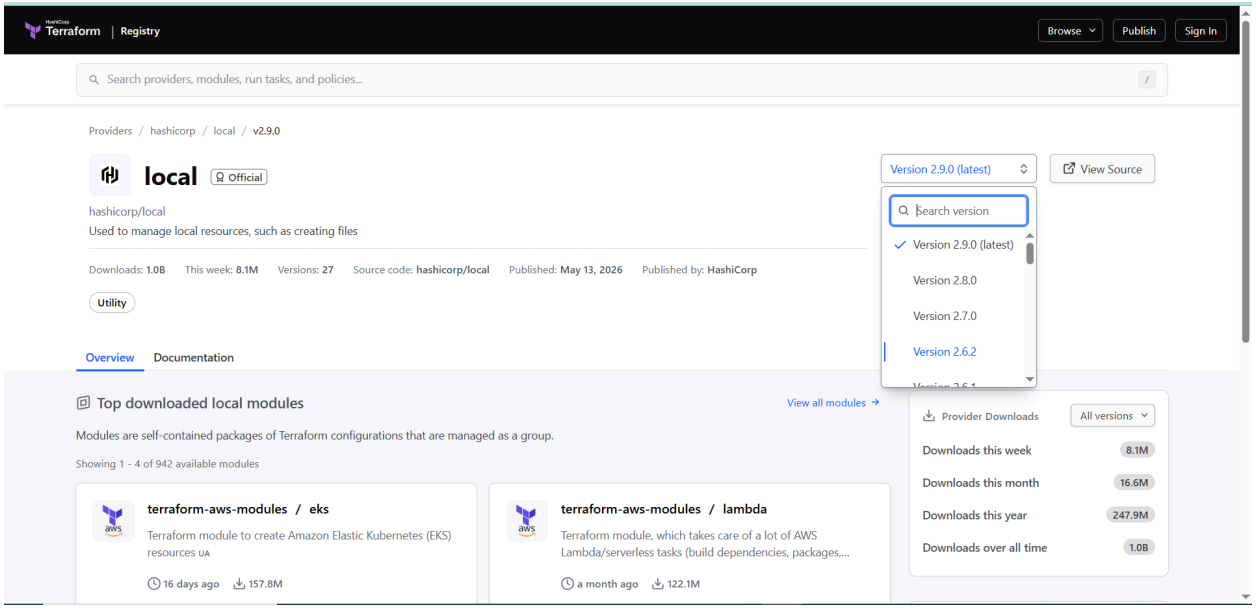
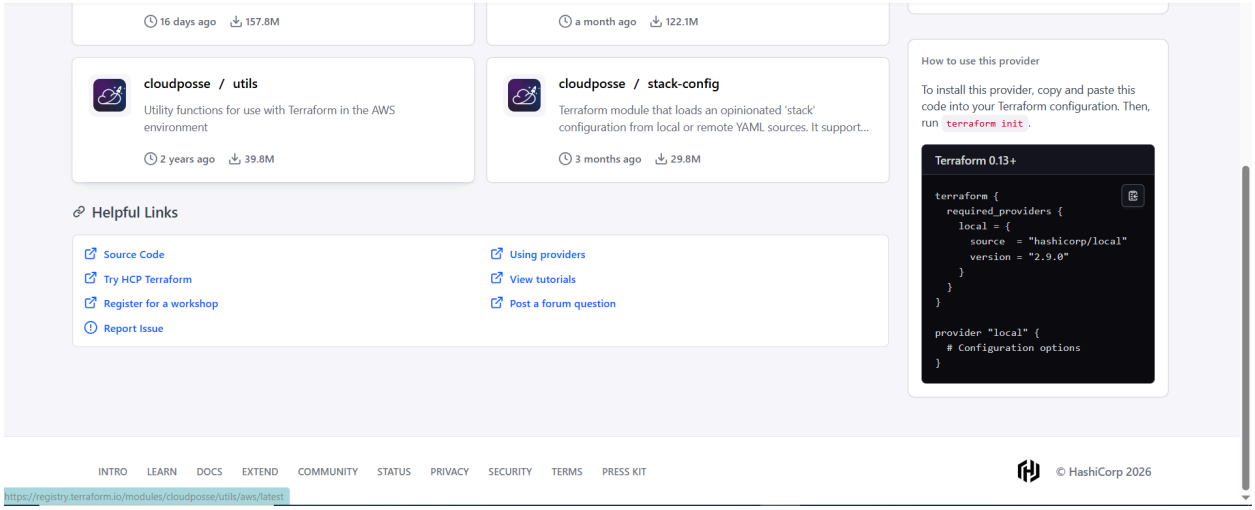
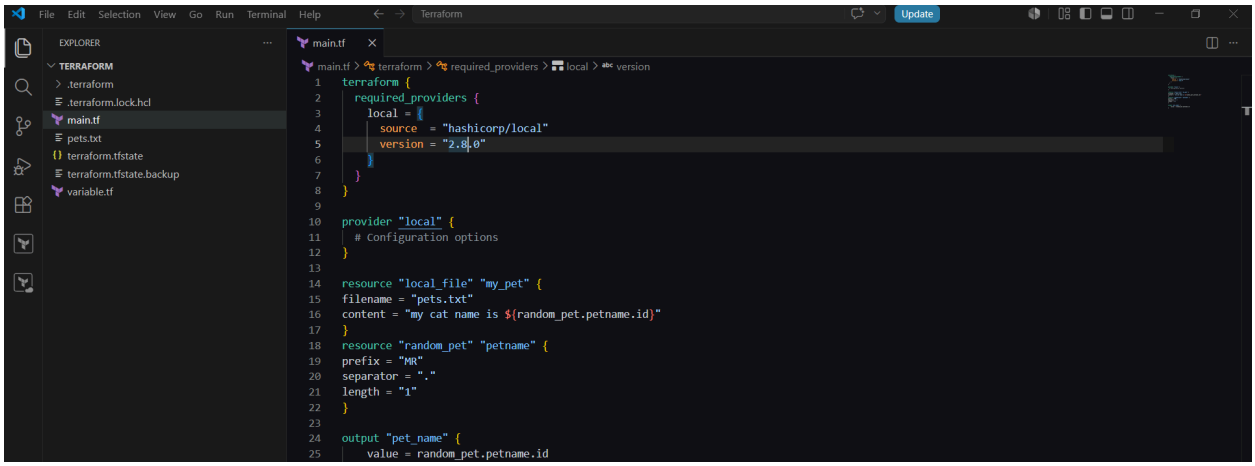
Terraform-04

1). Watch **Terraform-04** video.
2). Execute the script shown in the video.

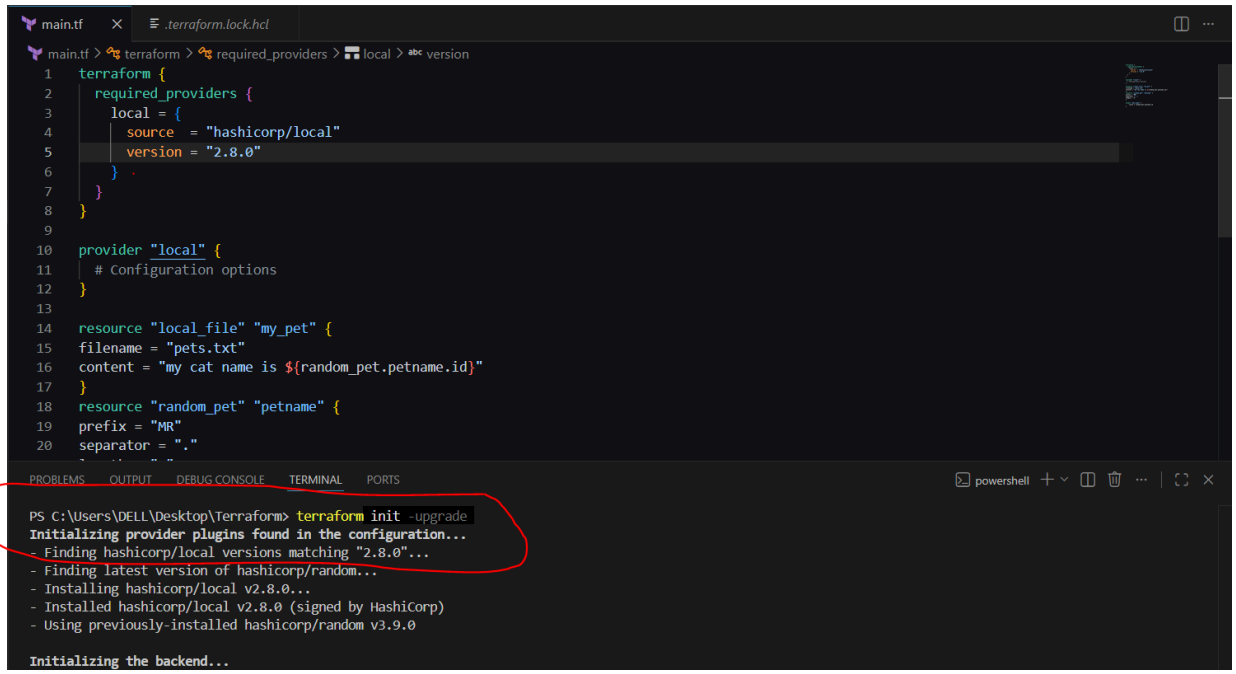
Version Constraints:
Changing in terraform providers version may get us into incompatibility issues.
By default terraform will always try to download the latest version of provider
available from the registry.
To make sure to use the specific version provider we can add the provider block in configuration.

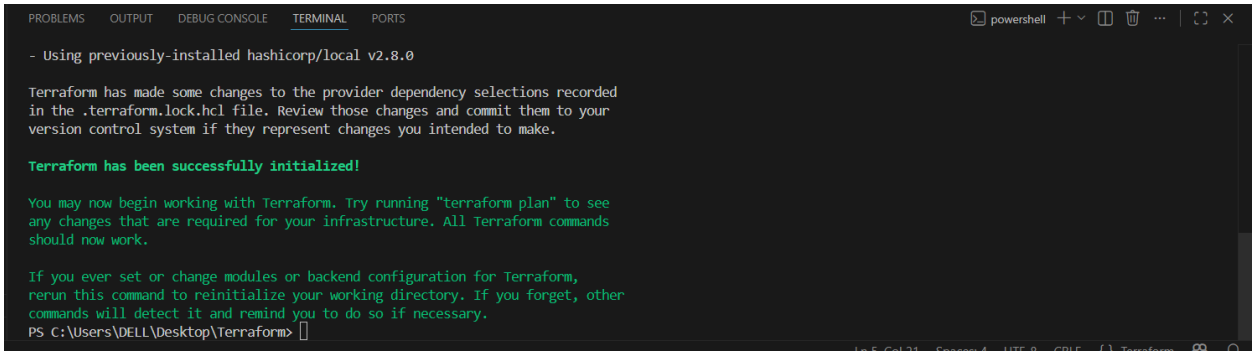


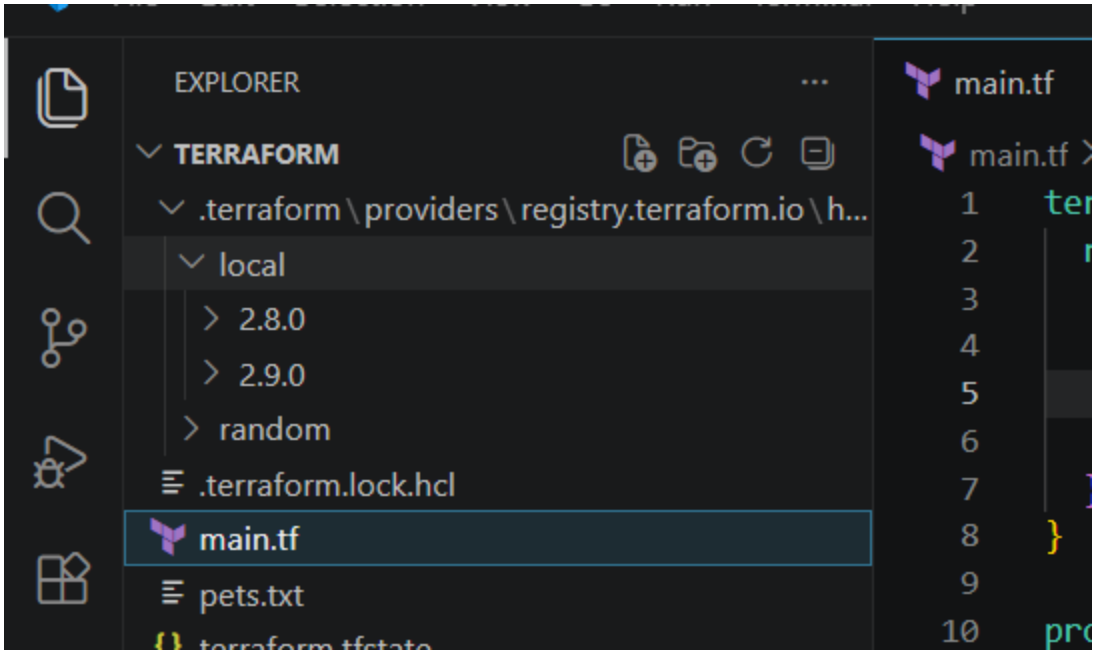

16 days ago 157.8M
a month ago 122.1M
cloudposse / utils
Utility functions for use with Terraform in the AWS environment
2 years ago 39.8M
cloudposse / stack-config
Terraform module that loads an opinionated 'stack' configuration from local or remote YAML sources. It support...
3 months ago 29.8M
How to use this provider
To install this provider, copy and paste this code into your Terraform configuration. Then, run terraform init.
Terraform 0.13+
terraform {
required_providers {
local = {
source = "hashicorp/local"
version = "2.9.0"
}
}
}
provider "local" {
# Configuration options
}
Helpful Links
Source Code
Try HCP Terraform
Register for a workshop
Report Issue
Using providers
View tutorials
Post a forum question
INTRO LEARN DOCS EXTEND COMMUNITY STATUS PRIVACY SECURITY TERMS PRESS KIT
© HashiCorp 2026
https://registry.terraform.io/modules/cloudposse/utils/aws/latest


File Edit Selection View Go Run Terminal Help
Terraform
Update
EXPLORER
TERRAFORM
.terraform
.terraform.lock.hcl
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.backup
variable.tf
main.tf
main.tf > terraform > required_providers > local > version
terraform {
required_providers {
local = {
source = "hashicorp/local"
version = "2.8.0"
}
}
}
provider "local" {
# Configuration options
}
resource "local_file" "my_pet" {
filename = "pets.txt"
content = "my cat name is ${random_pet.petname.id}"
}
resource "random_pet" "petname" {
prefix = "MR"
separator = "."
length = "1"
}
output "pet_name" {
value = random_pet.petname.id

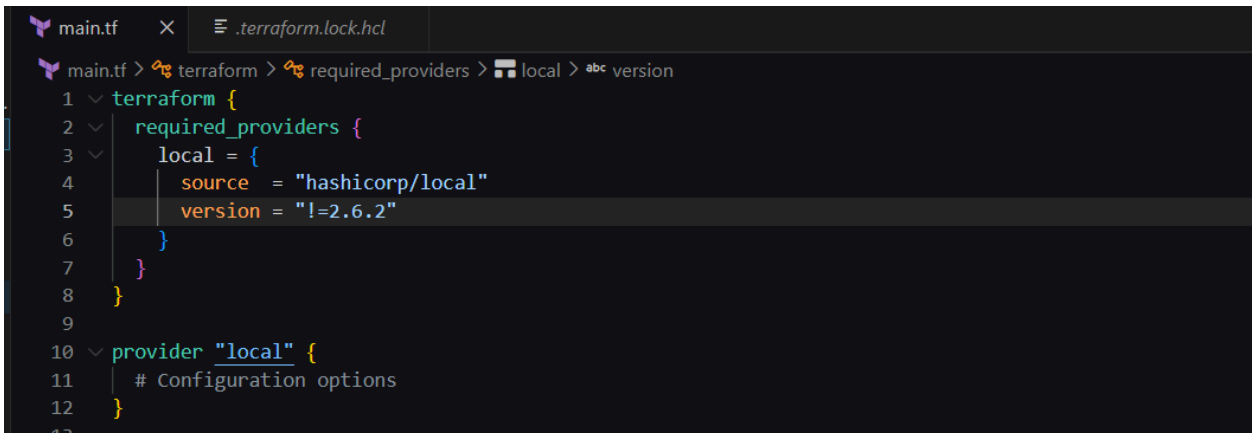
Upgrading from 2.9 to 2.8:

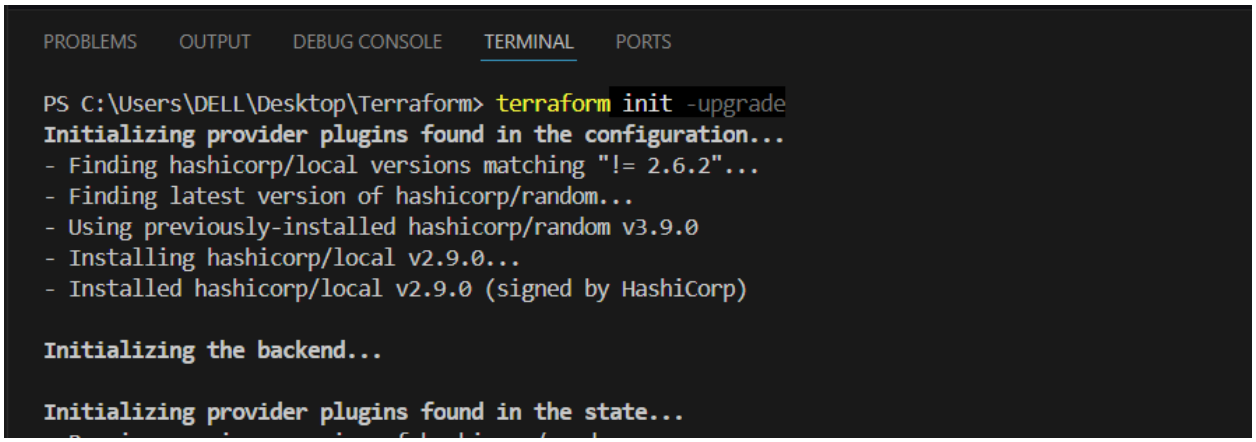




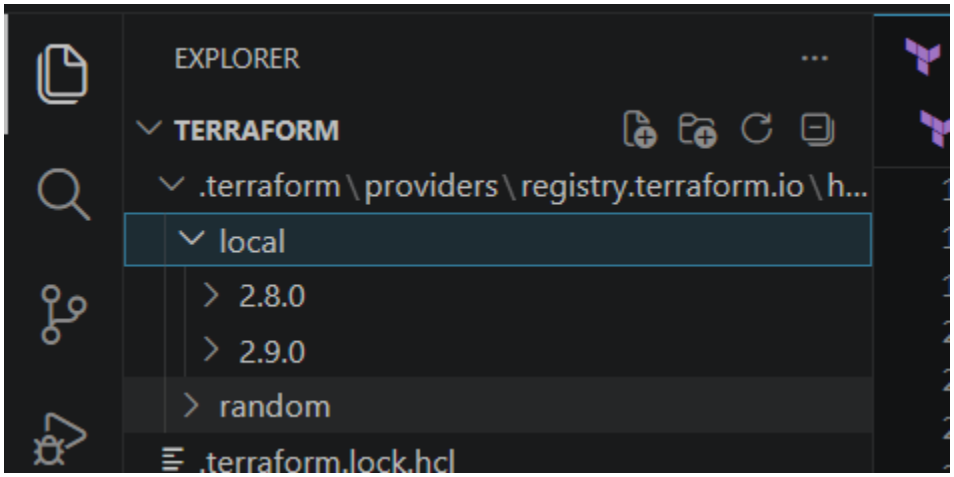

EXPLORER
TERRAFORM
.terraform \ providers \ registry.terraform.io \ h...
local
2.8.0
2.9.0
random
.terraform.lock.hcl
main.tf
pets.txt
main.tf
main.tf
1
2
3
4
5
6
7
8
9
10
}

version = "!=2.6.2"


main.tf
.terraform.lock.hcl
main.tf > terraform > required_providers > local > version
terraform {
required_providers {
local = {
source = "hashicorp/local"
version = "!=2.6.2"
}
}
}
provider "local" {
# Configuration options
}


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\DELL\Desktop\Terraform> terraform init -upgrade
Initializing provider plugins found in the configuration...
- Finding hashicorp/local versions matching "!= 2.6.2"...
- Finding latest version of hashicorp/random...
- Using previously-installed hashicorp/random v3.9.0
- Installing hashicorp/local v2.9.0...
- Installed hashicorp/local v2.9.0 (signed by HashiCorp)
Initializing the backend...
Initializing provider plugins found in the state...

```
- Using previously-installed hashicorp/local v2.9.0

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\DELL\Desktop\Terraform>
```



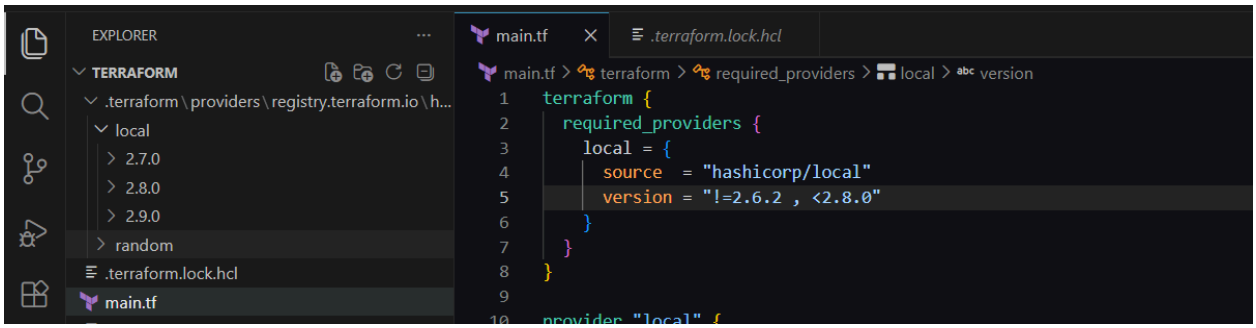
Passing 2 parameters:

main.tf .terraform.lock.hcl

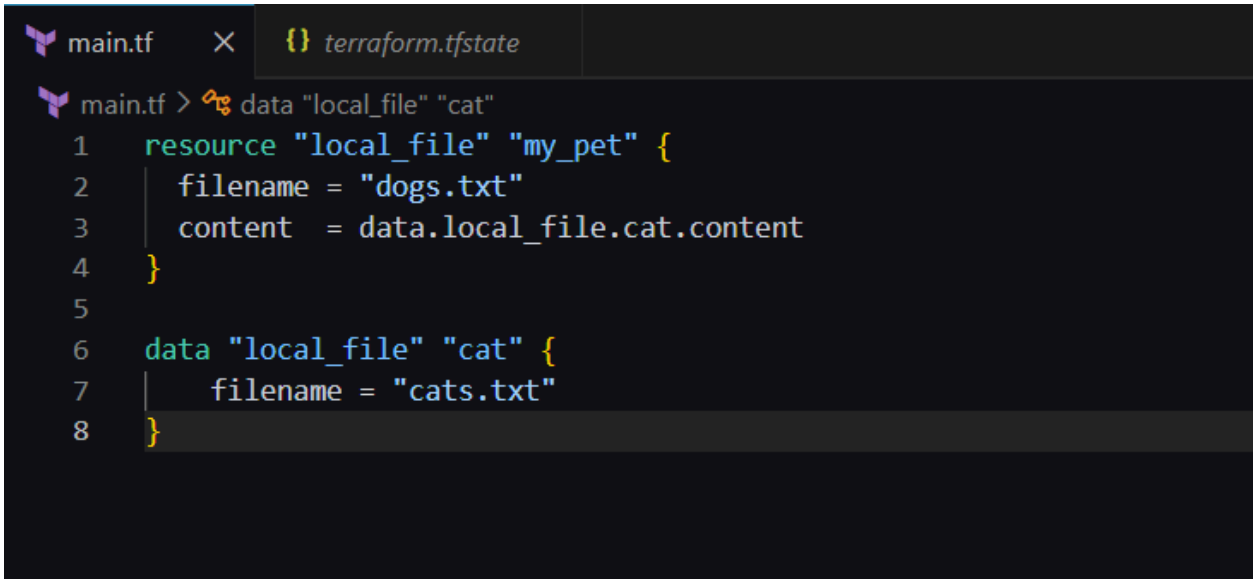
main.tf > terraform > required_providers > local > version

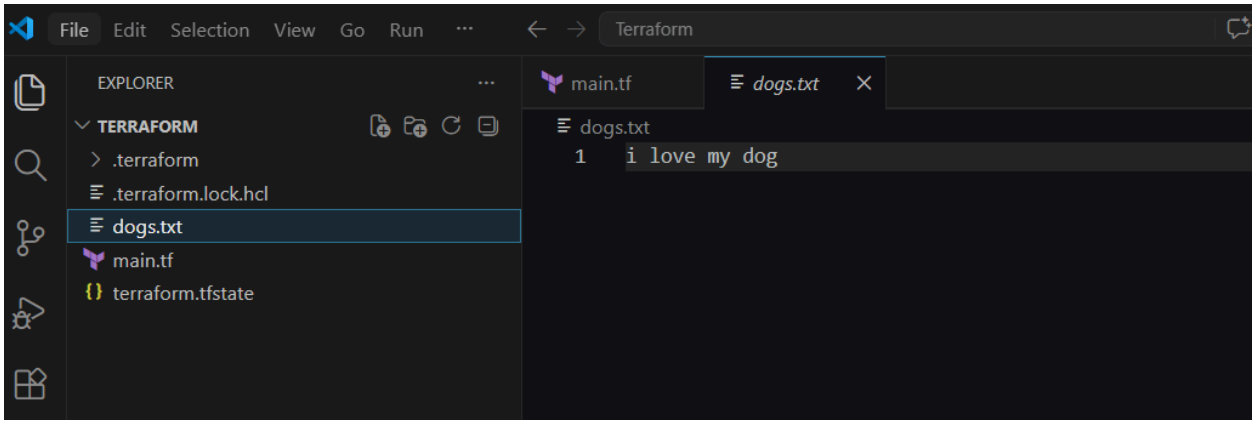
```
1   terraform {
2     required_providers {
3       local = {
4         source  = "hashicorp/local"
5         version = "!=2.6.2 , <2.8.0"
6       }
7     }
8   }
9
10  provider "local" {
11    # Configuration options
```

```
PS C:\Users\DELL\Desktop\Terraform> terraform init -upgrade
Initializing provider plugins found in the configuration...
- Finding hashicorp/local versions matching "!= 2.6.2, < 2.8.0"...
- Finding latest version of hashicorp/random...
- Installing hashicorp/local v2.7.0...
- Installed hashicorp/local v2.7.0 (signed by HashiCorp)
- Using previously-installed hashicorp/random v3.9.0

Initializing the backend...

Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/local
- Reusing previous version of hashicorp/random
- Using previously-installed hashicorp/local v2.7.0
- Using previously-installed hashicorp/random v3.9.0
```



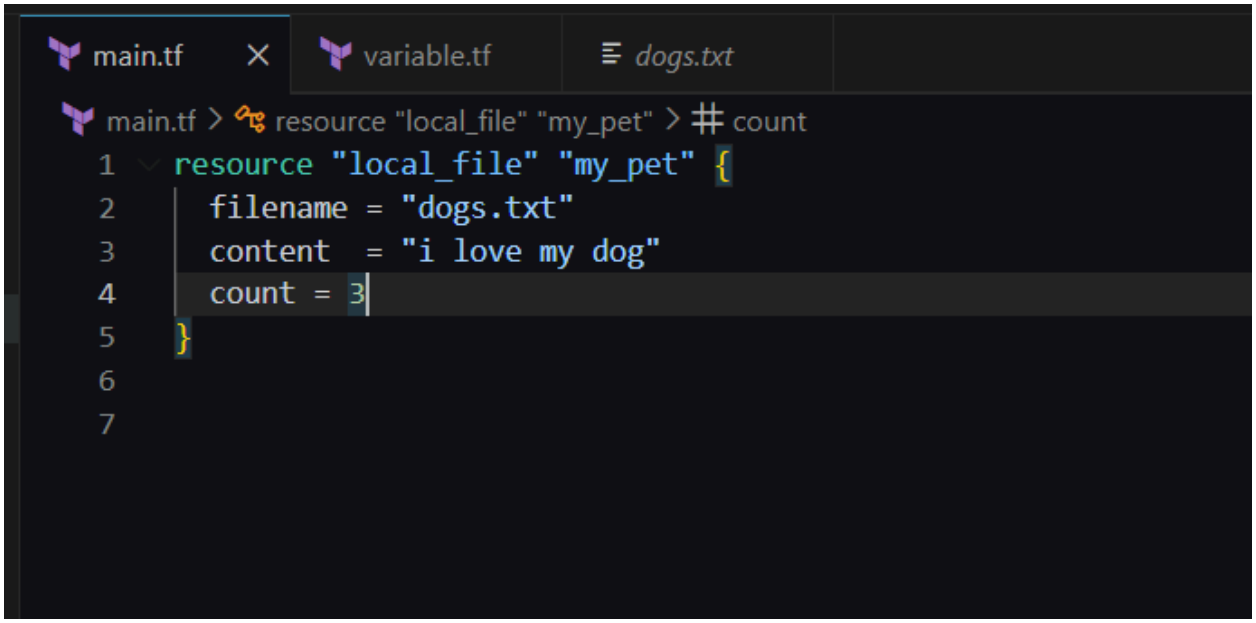
Data variables:



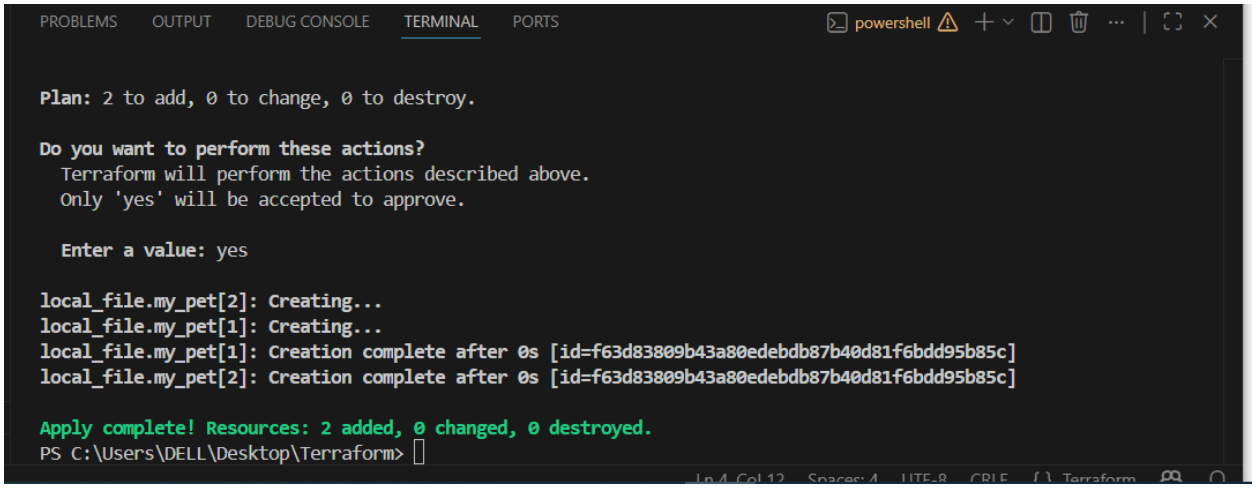


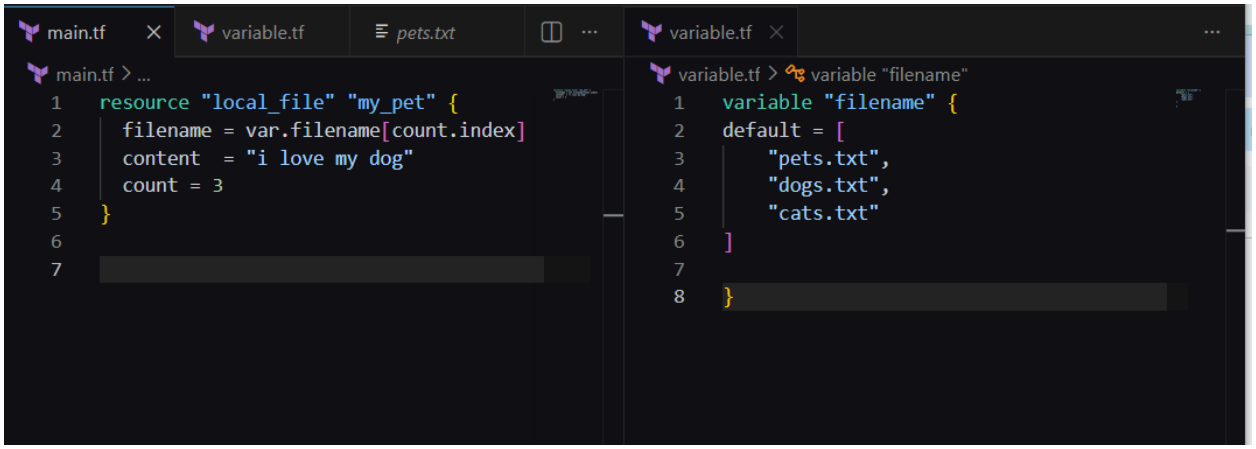
Now you can check in pets.txt
It is read from the main.tf and dogs.txt and then output will get in pets.txt

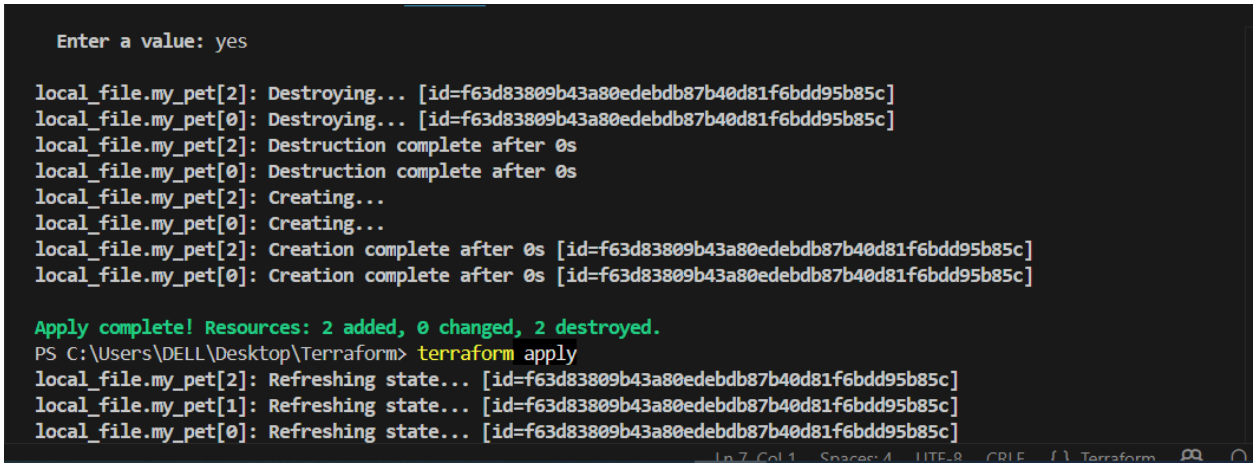
Meta arguments:



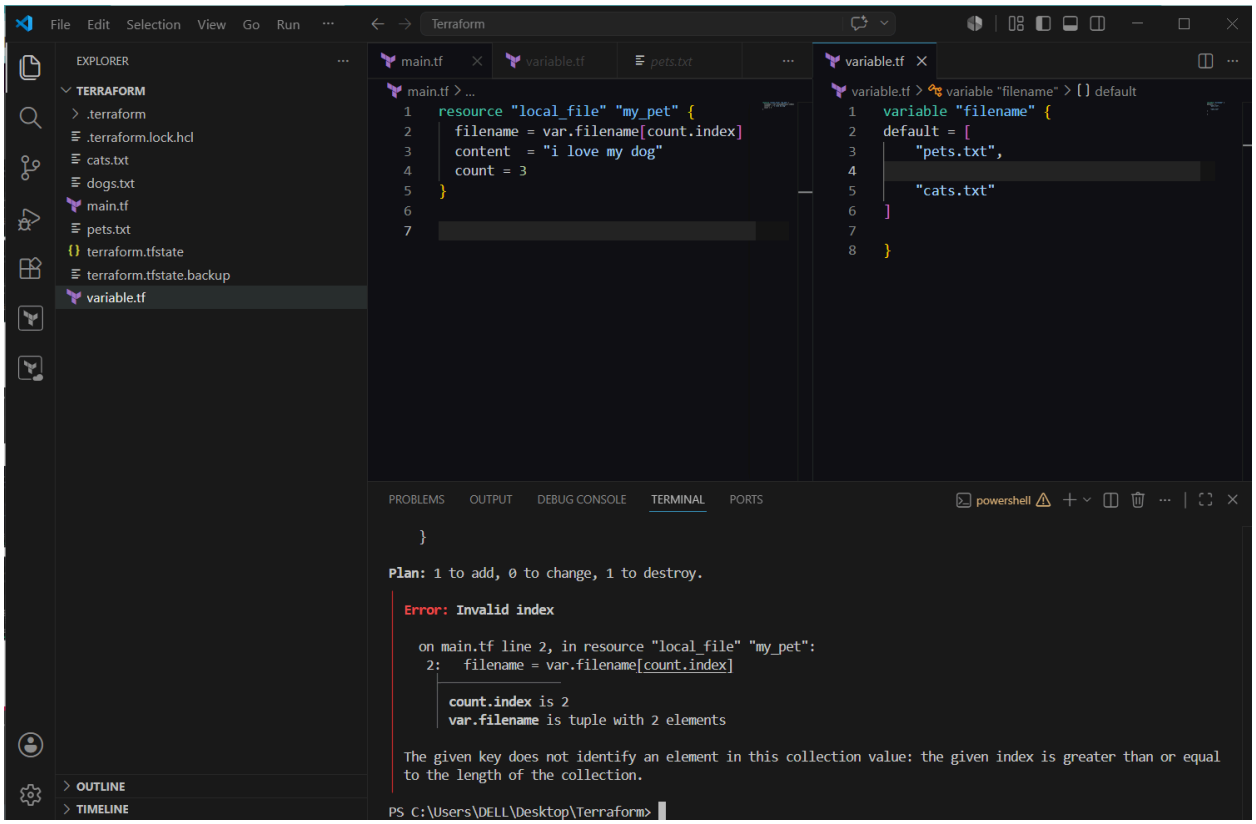
It is overriding:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
powershell
Plan: 2 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
local_file.my_pet[2]: Creating...
local_file.my_pet[1]: Creating...
local_file.my_pet[1]: Creation complete after 0s [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[2]: Creation complete after 0s [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
PS C:\Users\DELL\Desktop\Terraform>

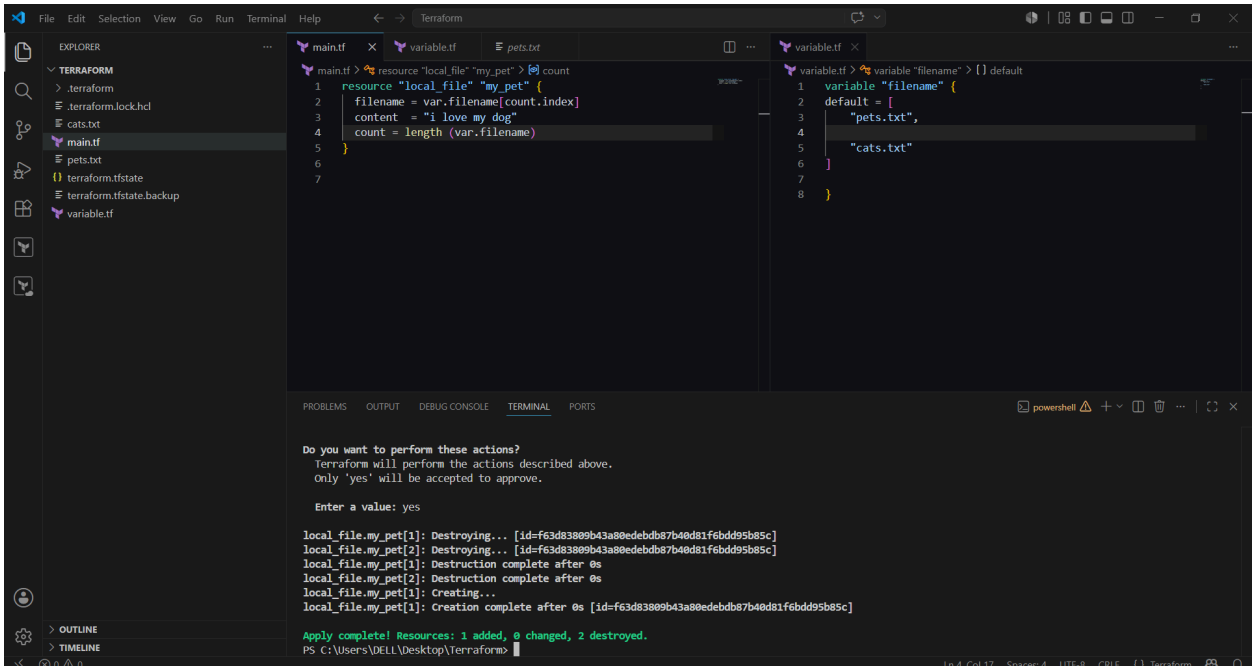

main.tf
variable.tf
pets.txt
main.tf > ...
resource "local_file" "my_pet" {
filename = var.filename[count.index]
content = "i love my dog"
count = 3
}
variable.tf
variable.tf > variable "filename"
variable "filename" {
default = [
"pets.txt",
"dogs.txt",
"cats.txt"
]
}


Enter a value: yes
local_file.my_pet[2]: Destroying... [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[0]: Destroying... [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[2]: Destruction complete after 0s
local_file.my_pet[0]: Destruction complete after 0s
local_file.my_pet[2]: Creating...
local_file.my_pet[0]: Creating...
local_file.my_pet[2]: Creation complete after 0s [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[0]: Creation complete after 0s [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
PS C:\Users\DELL\Desktop\Terraform> terraform apply
local_file.my_pet[2]: Refreshing state... [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[1]: Refreshing state... [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]
local_file.my_pet[0]: Refreshing state... [id=f63d83809b43a80edebdb87b40d81f6bdd95b85c]

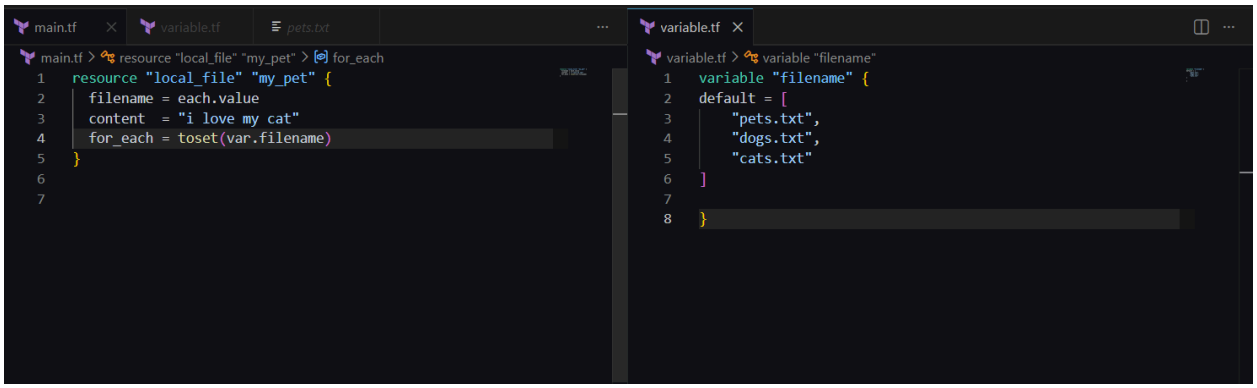
Here throughs an error because of in main.tf file count is 3 and in variable.tf we have 2 files, thats why throughs error

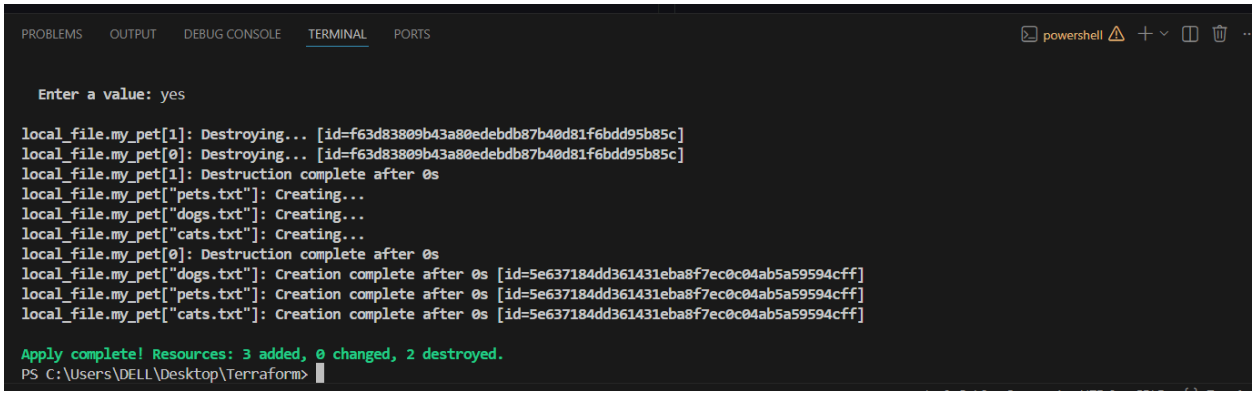


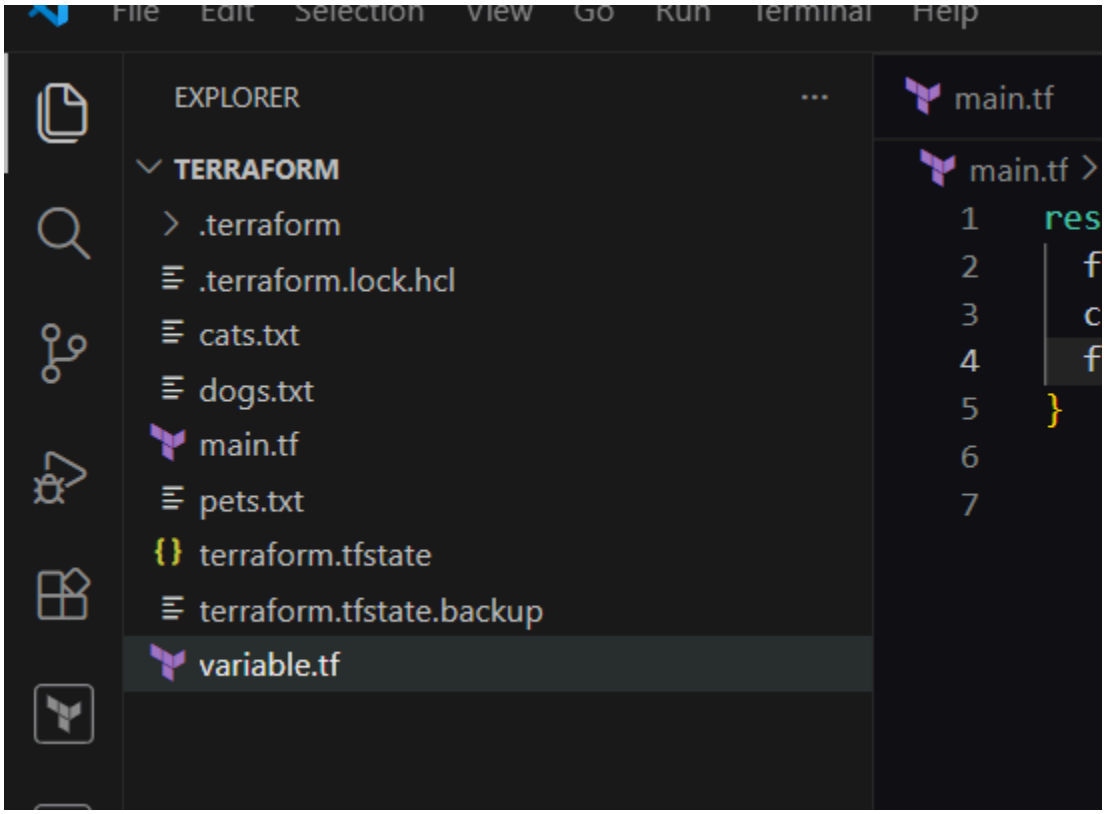
count = lenght(var.filename):

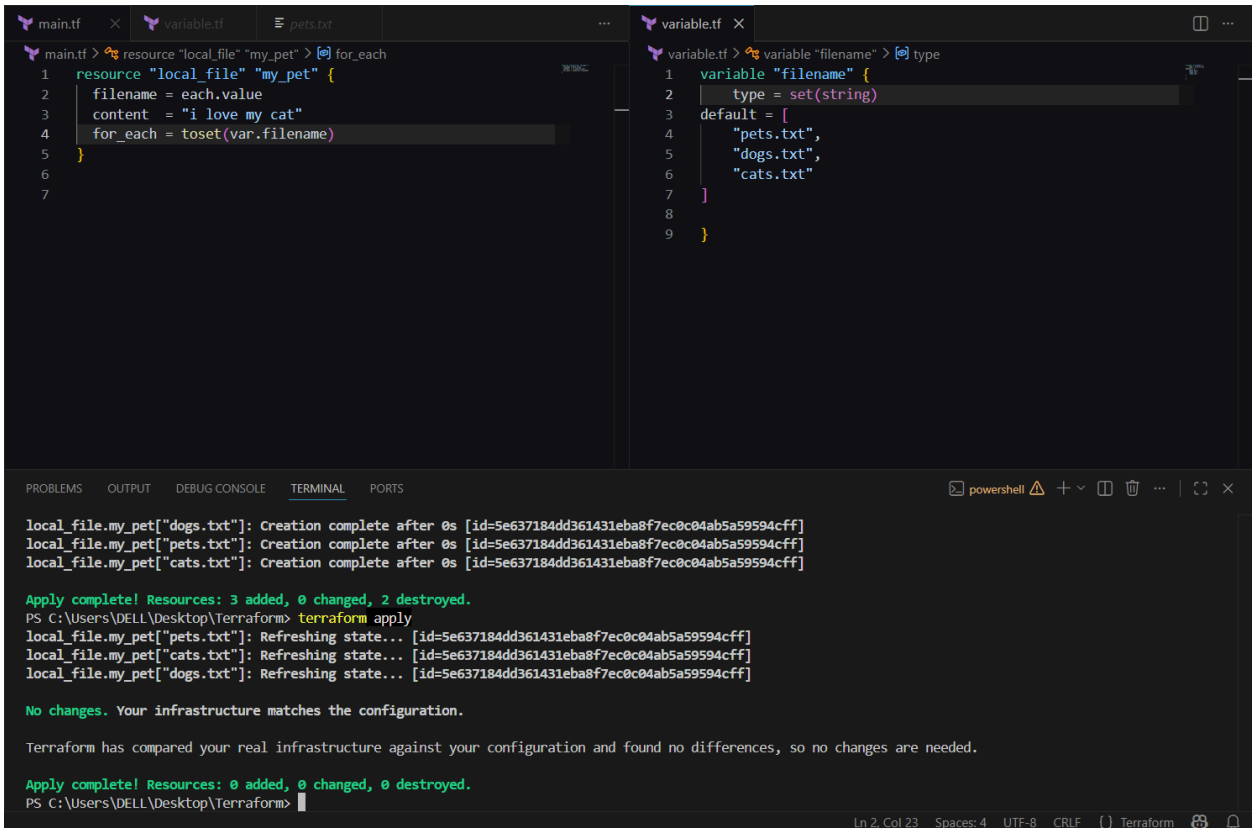


Using for each:






File Edit Selection View Go Run Terminal Help
EXPLORER
TERRAFORM
.terraform
.terraform.lock.hcl
cats.txt
dogs.txt
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.backup
variable.tf
main.tf
main.tf
1 res
2 f
3 c
4 f
5 }
6
7


main.tf
variable.tf
pets.txt
main.tf > resource "local_file" "my_pet" > for_each
1 resource "local_file" "my_pet" {
2 filename = each.value
3 content = "i love my cat"
4 for_each = toset(var.filename)
5 }
6
7
variable.tf
variable.tf > variable "filename" > type
1 variable "filename" {
2 type = set(string)
3 default = [
4 "pets.txt",
5 "dogs.txt",
6 "cats.txt"
7]
8
9 }
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
local_file.my_pet["dogs.txt"]: Creation complete after 0s [id=5e637184dd361431eba8f7ec0c04ab5a59594cff]
local_file.my_pet["pets.txt"]: Creation complete after 0s [id=5e637184dd361431eba8f7ec0c04ab5a59594cff]
local_file.my_pet["cats.txt"]: Creation complete after 0s [id=5e637184dd361431eba8f7ec0c04ab5a59594cff]
Apply complete! Resources: 3 added, 0 changed, 2 destroyed.
PS C:\Users\DELL\Desktop\Terraform> terraform apply
local_file.my_pet["pets.txt"]: Refreshing state... [id=5e637184dd361431eba8f7ec0c04ab5a59594cff]
local_file.my_pet["cats.txt"]: Refreshing state... [id=5e637184dd361431eba8f7ec0c04ab5a59594cff]
local_file.my_pet["dogs.txt"]: Refreshing state... [id=5e637184dd361431eba8f7ec0c04ab5a59594cff]
No changes. Your infrastructure matches the configuration.
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\Users\DELL\Desktop\Terraform>
Ln 2, Col 23 Spaces: 4 UTF-8 CRLF Terraform

Creating I am user in EC2 by using Terraform:

Aws configure:

```
PS C:\Users\DELL\Desktop\Terraform> aws configure
AWS Secret Access Key [****************eAv9]: /3zoV08WMQVzG/epY466Iq1PhvvFrTOuj4fDwITe
Default region name [ap-south-1]:
Default output format [json]:
PS C:\Users\DELL\Desktop\Terraform> aws -version
```

Write a script: creating IAM user

main.tf

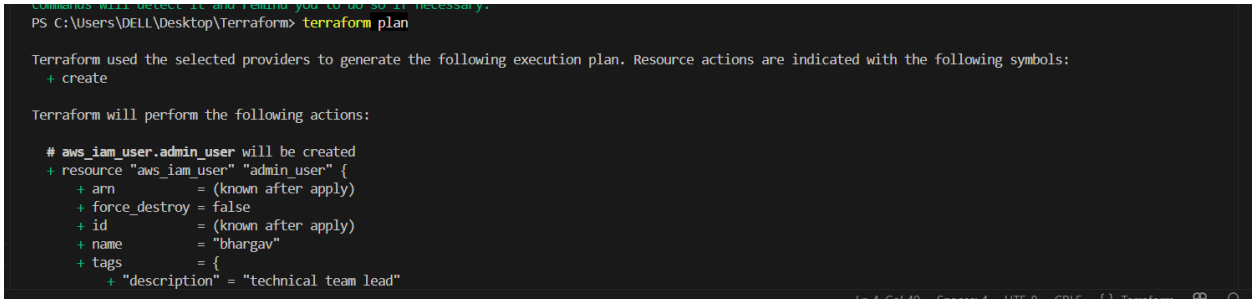
main.tf > resource "aws_iam_user" "admin_user" > tags > description

```
1  resource "aws_iam_user" "admin_user" {
2  name = "bhargav"
3  tags = {
4      description = "technical team lead"
5  }
6  }
```

Terraform init:

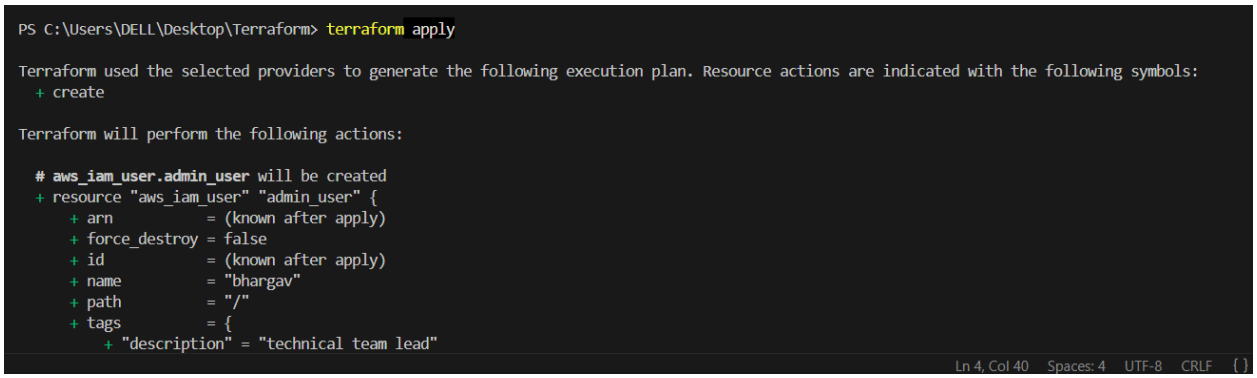
```
PS C:\Users\DELL\Desktop\Terraform> terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.50.0...
- Installed hashicorp/aws v6.50.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

Terrafirm plan:



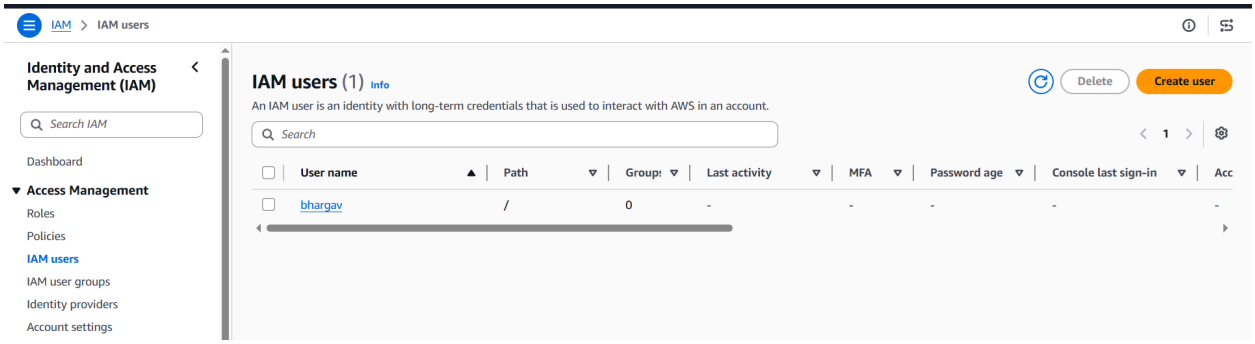
```
PS C:\Users\DELL\Desktop\Terraform> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_user.admin_user will be created
  + resource "aws_iam_user" "admin_user" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "bhargav"
      + tags          = {
          + "description" = "technical team lead"
```

Terraform apply:



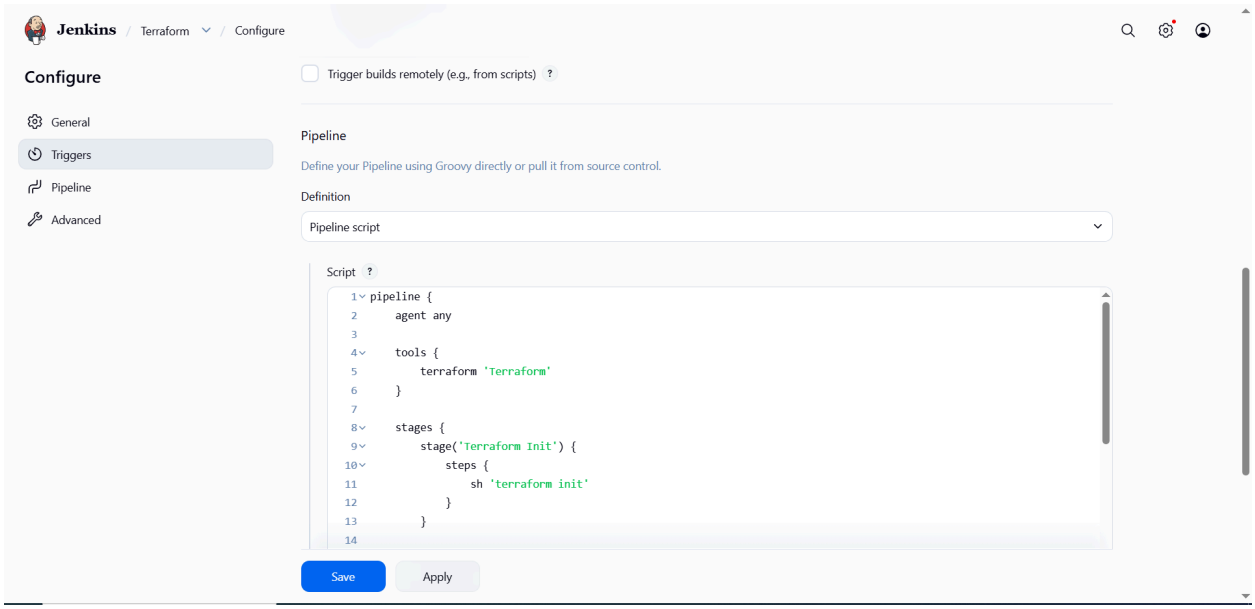
```
PS C:\Users\DELL\Desktop\Terraform> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_user.admin_user will be created
  + resource "aws_iam_user" "admin_user" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "bhargav"
      + path          = "/"
      + tags          = {
          + "description" = "technical team lead"
Ln 4, Col 40  Spaces: 4  UTF-8  CRLF  { }
```

Verify : automrtically crate a new user



3). Integrate **Terraform in Jenkins** using the **Terraform plugin**.

```
[root@ip-172-31-14-229 ~]# cd /var/lib/jenkins/workspace/Terraform
[root@ip-172-31-14-229 Terraform]# ls
main.tf  terraform-from-jenkins.txt  terraform.tfstate
[root@ip-172-31-14-229 Terraform]# cat main.tf
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "demo" {
  filename = "terraform-from-jenkins.txt"
  content  = "Terraform executed successfully from Jenkins using Terraform Plugin"
}
[root@ip-172-31-14-229 Terraform]#
```



```
pipeline {
    agent any

    tools {
        terraform 'Terraform'
    }

    stages {
        stage('Terraform Init') {
            steps {
                sh 'terraform init'
            }
        }

        stage('Terraform Plan') {
            steps {
                sh 'terraform plan'
            }
        }

        stage('Terraform Apply') {
            steps {
                sh 'terraform apply -auto-approve'
            }
        }
    }
}
```

4). Create a **CI/CD pipeline** for a Nodejs Application:
https://github.com/betawins/Trading-UI.git

Status of jenkins

```
[root@ip-172-31-14-229 ~]# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)
     Active: active (running) since Thu 2026-06-11 10:21:59 UTC; 4 days ago
   Main PID: 3032 (java)
      Tasks: 46 (limit: 4543)
     Memory: 1003.0M
        CPU: 21min 45.879s
     CGroup: /system.slice/jenkins.service
             └─3032 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --we

Jun 14 10:22:13 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-14 10:22:13.
Jun 14 10:22:13 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-14 10:22:13.
Jun 14 17:37:53 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-14 17:37:53.
Jun 15 10:22:01 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-15 10:22:01.
Jun 15 10:22:02 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-15 10:22:02.
Jun 15 10:22:04 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-15 10:22:04.
Jun 15 10:22:06 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-15 10:22:06.
Jun 15 10:22:07 ip-172-31-14-229.ap-south-1.compute.internal jenkins[3032]: 2026-06-15 10:22:07.
```

Install nodejs: dnf install nodejs -y

```
[root@ip-172-31-14-229 ~]# dnf install nodejs -y
Last metadata expiration check: 8:20:24 ago on Mon Jun 15 02:04:42 2026.
Dependencies resolved.
================================================================================
 Package                    Architecture          Version
================================================================================
Installing:
 nodejs                     x86_64                1:18.20.8-1.amzn2023.0.2
Installing dependencies:
 nodejs-libs                x86_64                1:18.20.8-1.amzn2023.0.2
Installing weak dependencies:
 nodejs-docs                noarch                1:18.20.8-1.amzn2023.0.2
 nodejs-full-i18n           x86_64                1:18.20.8-1.amzn2023.0.2
 nodejs-npm                 x86_64                1:10.8.2-1.18.20.8.1.amzn2023.0.2

Transaction Summary
================================================================================
Install  5 Packages

Total download size: 45 M
Installed size: 223 M
Downloading Packages:
(1/5): nodejs-docs-18.20.8-1.amzn2023.0.2.noarch.rpm
(2/5): nodejs-18.20.8-1.amzn2023.0.2.x86_64.rpm
(3/5): nodejs-full-i18n-18.20.8-1.amzn2023.0.2.x86_64.rpm
```

verify:

```
[root@ip-172-31-14-229 ~]# node -v
v18.20.8
```

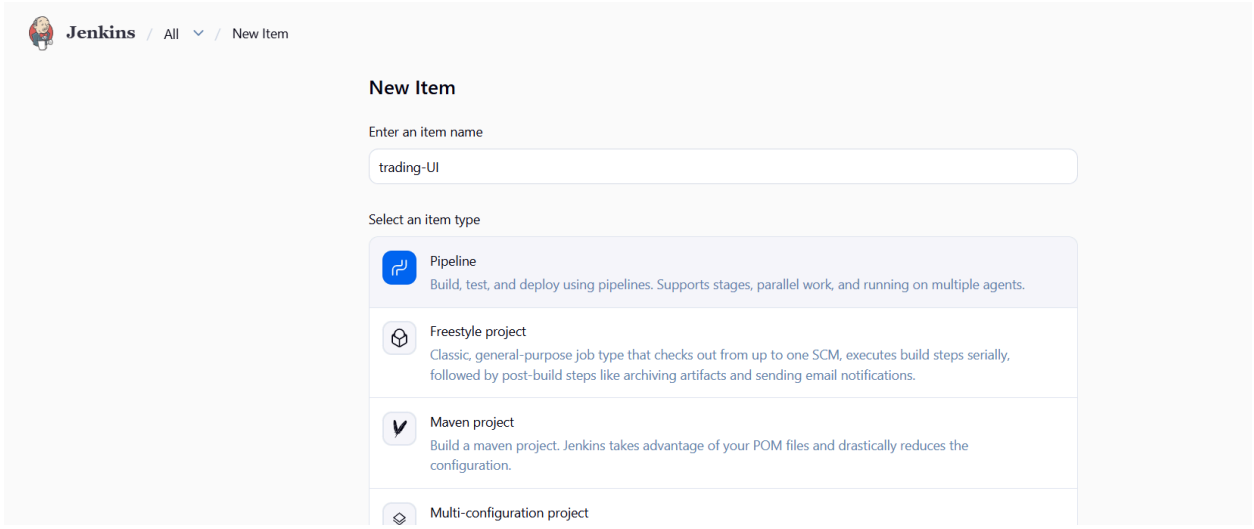
Install pm2: npm install -g pm2

```
[root@ip-172-31-14-229 ~]# npm install -g pm2

added 90 packages in 4s

8 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New major version of npm available! 10.8.2 -> 11.17.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.17.0
npm notice To update run: npm install -g npm@11.17.0
npm notice
```

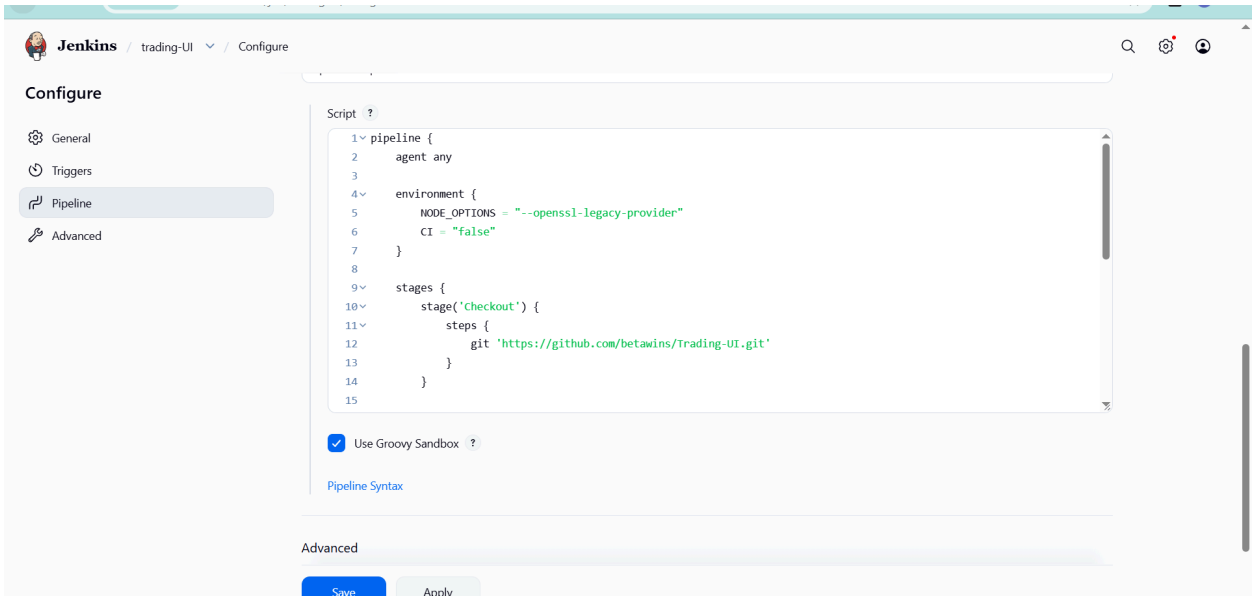
verify:

```
[root@ip-172-31-14-229 ~]# pm2 -v

                     -------------

__/\\\\\\\\\\\\\____/\\\\____________/\\\\____/\\\\\\\\\_____
 _\/\\\/////////\\\_\/\\\\\\________/\\\\\\__/\\\/////////\\\___
  _\/\\\_______\/\\\_\/\\\//\\\____/\\\//\\\_\///______\//\\\__
   _\/\\\\\\\\\\\\\/__\/\\\\///\\\/\\\/_\/\\\___________/\\\/___
    _\/\\\/////////____\/\\\__\///\\\/___\/\\\\\\\\\_____/\\\//_____
     _\/\\\_____________\/\\\____\///_____\/\\\////\\\___/\\\//________
      _\/\\\_____________\/\\\_____________\/\\\__\//\\\_/\\\/___________
       _\/\\\_____________\/\\\_____________\/\\\___\///\\\\\\\\\\\\\\\_
        _\///______________\///______________\///______\///////////////__


                      Runtime Edition

       PM2 is a Production Process Manager for Node.js applications
                    with a built-in Load Balancer.

              Start and Daemonize any application:
              $ pm2 start app.js

              Load Balance 4 instances of api.js:
```

Create new item:



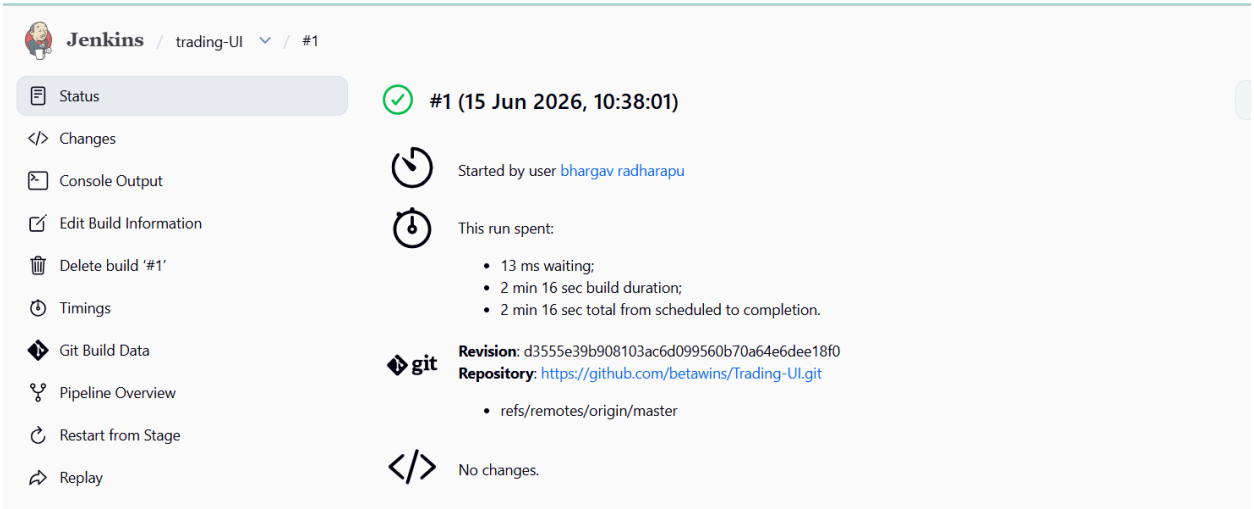
Write a pipeline:



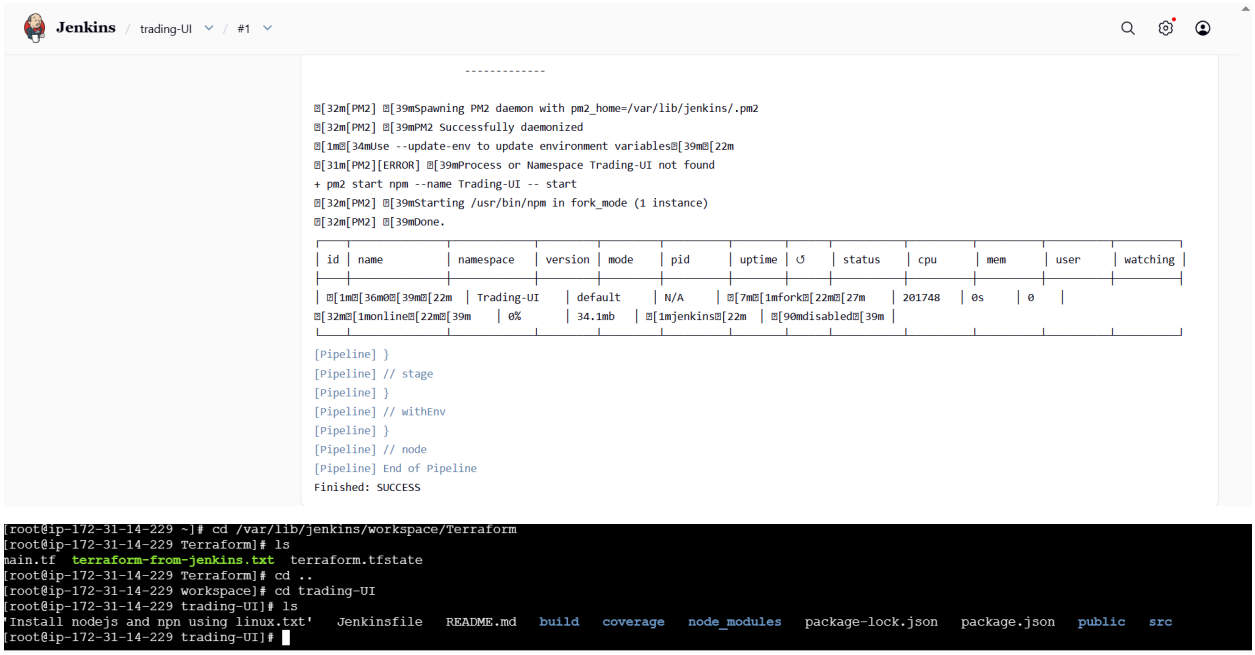

```
pipeline {
    agent any

    environment {
        NODE_OPTIONS = "--openssl-legacy-provider"
        CI = "false"
    }

    stages {
        stage('Checkout') {
            steps {
                git 'https://github.com/betawins/Trading-UI.git'
            }
        }

        stage('Install & Build') {
            steps {
                sh '''
                npm install
                npm run build
                '''
```

```
        }
    }

    stage('Deploy') {
        steps {
            sh '''
            pm2 restart Trading-UI || pm2 start npm --name Trading-UI --
start
            '''
        }
    }
  }
}
```

Save and build
Build succesfull



Verify output:



5). Explain **10 Maven commands**

Apache Maven is a popular Java build and dependency management tool. Its
commands are run through the mvn CLI. Here are 10 important Maven commands and what they do:

1. mvn clean

Removes the target/ directory where compiled files and build outputs are stored. Useful to ensure a fresh build without old artifacts interfering.

2. mvn compile

Compiles the source code of your project (src/main/java). It does not run tests
or package the application—only checks and compiles code.

3. mvn test

Runs unit tests defined in src/test/java using testing frameworks like JUnit. It
compiles both main code and test code before execution.

4. mvn package

Takes compiled code and packages it into a distributable format like a JAR or
WAR file inside the target/ directory.

5. mvn install

Installs the packaged artifact into your local Maven repository (~/.m2/repository) so it can be used as a dependency in other local projects.

6. mvn deploy

Copies the final package to a remote repository (like Nexus or Artifactory), making it available for other developers or CI/CD pipelines.

7. mvn validate

Checks whether the project structure and configuration are correct before the
build starts. It ensures the project is ready to build.

8. mvn verify

Runs additional checks on the package after tests, such as integration tests or
quality checks, to ensure the build is valid.

9. mvn dependency:tree

Displays the full dependency hierarchy of the project. Helpful for debugging version conflicts or unwanted transitive dependencies.

10. mvn archetype:generate

Creates a new Maven project from a template (archetype). It helps quickly scaffold standard project structures